

Megaprompt — NICOLÁS · Carrito, checkout y pedidos

Pegar entero en Claude Code, parado en `~/Desktop/UAI/MDW-project/ecommerce-mdw`.

Sos el desarrollador a cargo del **carrito, el checkout y los pedidos** del e-commerce de BC Importados, el trabajo final de Metodologías y Desarrollo Web (UAI). Trabajás con otras tres personas: Yasmín (identidad y RBAC), Genaro (catálogo) y Agustín (kernel, infraestructura, inventario y pagos).

Hoy es domingo 04/10/2026. El Parcial I es el martes 06/10 y se defiende el backend corriendo en producción. Si la URL pública no responde o el login no funciona, el parcial no se toma y pasa a la recuperación del 24/11.

Tu módulo es **el que decide si esto es un producto o una planilla con botones**. El núcleo obligatorio de la materia pide *un flujo de negocio real de punta a punta, no sólo ABMs*, y la cátedra lo dice sin vueltas:

Si al terminar su API tiene solamente los cuatro verbos por cada entidad, todavía no modelaron el negocio. La operación que no es crear, leer, modificar ni borrar —emitir, aprobar, reservar, despachar— es la que se defiende en el Parcial I.

Esas operaciones son tuyas: **crear el pedido, despacharlo, entregarlo y cancelarlo**.

Estado real del repositorio

- Ya existe `backend/config/viper.go`, que es el **único** lugar del backend donde se leen variables de entorno (ADR-001). Ya está declarada `ORDER_ACCESS_TOKEN_SECRET`, que es la que firma el enlace privado del invitado.
- Agustín está subiendo el **kernel**: `cmd/api/main.go`, la conexión a Postgres, el migrador y el paquete `apierr`. Esperá a que esté mergeado en `develop` antes de escribir handlers; el dominio, la migración y las reglas puras podés empezarlas ya.
- Yasmín está subiendo el **middleware de sesión y permisos**.
- Genaro está subiendo el **lector de catálogo**.
- `docs/specs.md` tiene tus historias: `SPEC-H07` (carrito), `SPEC-H13` (cotizar entrega y crear el pedido), `SPEC-H15` (efectivo con retiro), `SPEC-H16` (consultar el pedido) y `SPEC-H17` (preparación y entrega). La sección 5 tiene el flujo principal paso a paso y la sección 6 las reglas de estados.

Decisiones ya tomadas que tenés que respetar

- Go 1.26 · Chi · GORM · gormigrate · PostgreSQL (ADR-001, 002 y 003).
- Plata en `int64` de centavos**, columnas `bigint`, campos `..._centavos`. Nunca `float64`.
- Identificadores `uuid.UUID`**, columnas `uuid`. Nunca autoincremental: con un autoincremental cualquiera adivina la URL del pedido del vecino sumando uno.
- Nombres de tablas y columnas en castellano y `snake_case`, como el ERD.
- Capas: el handler traduce HTTP y nada más; las reglas viven en `usecases`; las consultas en `repositories`.

Contratos de los otros módulos (ya congelados, programá contra ellos)

```
// Genaro – backend/internal/usecases/catalogo
type VarianteVendible struct {
    ID uuid.UUID; ProductoID uuid.UUID;Codigo string; Nombre string
    PrecioMinoristaCentavos int64; Activa bool
}
func NuevoLector(db *gorm.DB) *Lector
func (l *Lector) VariantePorID(ctx context.Context, id uuid.UUID) (VarianteVendible, error)

// Agustín – backend/internal/usecases/inventario
type ItemReserva struct { VarianteID uuid.UUID; Unidades int }
func NuevoServicioStock(db *gorm.DB) *ServicioStock
func (s *ServicioStock) Disponible(ctx context.Context, varianteID uuid.UUID) (int, error)
func (s *ServicioStock) Reservar(ctx, tx *gorm.DB, pedidoID uuid.UUID, items []ItemReserva) error
func (s *ServicioStock) Confirmar(ctx, tx *gorm.DB, pedidoID uuid.UUID) error
func (s *ServicioStock) Liberar(ctx, tx *gorm.DB, pedidoID uuid.UUID) error

// Yasmín – backend/internal/adapters/rest/middleware
func (m *Middleware) RequiereSesion(next http.Handler) http.Handler
func (m *Middleware) RequierePermiso(p string) func(http.Handler) http.Handler
func (m *Middleware) SesionOpcional(next http.Handler) http.Handler
func UsuarioDeContexto(ctx context.Context) (Usuario, bool)
```

Si alguno todavía no está en `develop`, declarálo como **interfaz propia** en `backend/internal/core/ports/out/` y trabajá contra un stub en memoria. Es exactamente para eso que el proyecto es hexagonal.

Reglas duras

1. **Sólo tocás los archivos que esta consigna te asigna.** No edites archivos de Yasmín, Genaro ni Agustín. `go.mod`, `main.go`, `router.go` y `registry.go` son de Agustín.
2. `go build ./...` y `go vet ./...` en verde antes de cada push.
3. **El precio nunca sale del body.** Se lo pedís al lector de Genaro. Si el precio viniera del body, cualquiera compra a \$1 desde Postman.
4. **La identidad nunca sale del body.** Sale del contexto de la sesión o del token del enlace de invitado.
5. **Toda consulta de pedidos lleva el id del comprador en el WHERE.** No un `if` comparando ids después de traer la fila: para cuando hacés el `if`, el pedido ajeno ya está en memoria del servidor y cualquier `return` que se cuele antes lo filtra.
6. Cada commit en español, en imperativo, y es una pieza que compila. Al terminar cada bloque: `git add`, `git commit`, `git push`. **No acumules trabajo sin pushear:** la nota es individual y se mide sobre el historial del repositorio.
7. Rama: `feat/pedidos`, salida de `develop`. `main` está protegida.

```
git fetch --all
git checkout develop && git pull
git checkout -b feat/pedidos
```

BLOQUE 1 — Dominio y migración

1.1 Entidades — `backend/internal/core/domain/pedidos.go`

Carrito	id, usuario_id (único, nullable para invitado), creado_en, actualizado_en
ItemCarrito	id, carrito_id, variante_id, unidades, creado_en
Pedido	id, numero (secuencial legible, único), usuario_id (nullable: invitado), comprador_nombre, comprador_apellido, comprador_correo, comprador_telefono, comprador_dni, modo_entrega (RETIRO ENVIO), domicilio (nullable), distancia_metros (nullable), envio_centavos, subtotal_centavos, total_centavos, estado_pedido (PENDIENTE_DE_PAGO CONFIRMADO DESPACHADO COMPLETADO CANCELADO), estado_pago (PENDIENTE APROBADO RECHAZADO), medio_pago (MERCADO_PAGO EFECTIVO), token_acceso (para el enlace del invitado), vence_en (nullable), creado_en, actualizado_en
DetallePedido	id, pedido_id, variante_id, codigo, nombre, ← copia histórica unidades, precio_unitario_centavos, subtotal_centavos
ConfiguracionTienda	id, efectivo_habilitado, distancia_maxima_metros, creado_en
TarifaDistancia	id, configuracion_id, desde_metros, hasta_metros, costo_centavos

Dos cosas que son el corazón del módulo y se preguntan:

- **DetallePedido es la relación N-N con datos propios** entre `Pedido` y `Variante`. No es una tabla técnica: tiene unidades y precio, que es el trabajo de modelado de verdad. Es la N-N que exige el núcleo obligatorio, junto con las de Yasmín y Genaro.
- **codigo, nombre y precio_unitario_centavos se copian en el detalle.** Parece violar "cada dato en un solo lugar", y no lo hace: no es una copia del catálogo, es **lo que el comprador compró ese día**. Si mañana suben el precio o renombran el producto, el pedido viejo tiene que seguir diciendo lo que decía. La spec lo llama "copia inmutable" y es una regla de negocio, no una desnormalización.

`estado_pedido`, `estado_pago`, `modo_entrega` y `medio_pago` son listas cerradas: van como enum de Postgres o `varchar` con `CHECK`, nunca `varchar` libre.

1.2 Migración — `migrations/schema/pedidos.go`

Agustín dejó un stub con `func MigracionesPedidos() []*gormigrate.Migration` devolviendo vacío. Reemplazó su contenido. **Ese archivo es tuyo a partir de ahora.**

Una sola migración, ID `20261004_04_pedidos`. Corre última porque tiene FKs a `usuario`, `variante` y porque las reservas de Agustín apuntan a `pedido`.

Obligatorio:

- Único en `pedido(numero)` y en `pedido(token_acceso)`.

- Índices en `pedido(usuario_id)`, `pedido(estado_pedido)`, `pedido(creado_en)`, `detalle_pedido(pedido_id)`, `item_carrito(carrito_id)`, `carrito(usuario_id)`. `pedido(usuario_id)` es la FK por la que se filtra en **cada** consulta de "mis pedidos": sin índice, cada pantalla recorre la tabla entera.
- `onDelete` **explícito en cada FK, con el motivo en un comentario**:
 - `pedido → usuario`: `RESTRICT`. Un usuario con pedidos no se borra, se desactiva. El historial de ventas no se va con la baja de una cuenta.
 - `detalle_pedido → pedido`: `RESTRICT`. **Los pedidos no se borran, se cancelan.**
 - `detalle_pedido → variante`: `RESTRICT`. Un producto vendido no se borra del catálogo.
 - `item_carrito → carrito`: `CASCADE` (el carrito es una selección editable, no historial).

```
git add backend/internal/core/domain/pedidos.go \
    backend/internal/adapters/db/postgresql/migrations/schema/pedidos.go
git commit -m "feat(pedidos): entidades de carrito, pedido y detalle con copia historica y su migracion"
git push -u origin feat/pedidos
```

1.3 Repositorios — `repositories/pedidos/`

- `carritos.go`: `PorUsuario`, `CrearSiNoExiste`, `AgregarItem`, `CambiarUnidades`, `QuitarItem`, `Vaciar`.
- `pedidos.go`: `Crear(tx, pedido, detalles)`, `PorIDYUsuario(id, usuarioID)`, `PorToken(token)`, `ListarDeUsuario(usuarioID, limit, offset)`, `ListarAdmin(filtros, limit, offset)`, `CambiarEstado(tx, id, nuevo)`.
- `configuracion.go`: `Actual()`, `TarifasOrdenadas()`.

`PorIDYUsuario` es la consulta que define la seguridad del módulo:

```
// ✓ las dos condiciones en el WHERE
db.Where("id = ? AND usuario_id = ?", id, usuarioID).First(&p)
```

No es que el pedido ajeno se rechace después: **es que no existe para esa llamada**. El 404 sale solo, sin decidir nada, y no distingue "no existe" de "no es tuyo", que es justo lo que queremos. Y si mañana aparece otro endpoint, reusa la función y hereda la protección.

```
git add backend/internal/adapters/db/postgresql/repositories/pedidos/
git commit -m "feat(pedidos): repositorios con el id del comprador en el where de cada consulta"
git push
```

BLOQUE 2 — Las reglas puras (hacelas antes que los handlers)

`backend/internal/usecases/pedidos/`. **Sin importar GORM, sin importar `net/http` y sin llamar a `time.Now()` adentro**: todo entra por parámetro. Una función que mira el reloj no se puede testear —el test "la reserva vencida aparece vencida" tendría que esperar a que venza— y no se puede reusar para preguntar "¿esto era válido el 3 de octubre?".

2.1 `totales.go`

```
type LineaCalculada struct {
    VarianteID          uuid.UUID
    Codigo, Nombre      string
    Unidades            int
    PrecioUnitarioCentavos int64
    SubtotalCentavos    int64
}

func CalcularLineas(items []ItemPedido, precios map[uuid.UUID]VarianteVendible) ([]LineaCalculada, error)
func CalcularTotal(lineas []LineaCalculada, envioCentavos int64) (subtotal, total int64)
```

Todo en enteros. El subtotal de una línea es `unidades * precioUnitario`, el subtotal del pedido es la suma, y el total es `subtotal + envio`. Sin promociones ni cupones: quedan fuera del alcance del Parcial I.

2.2 `entrega.go`

```
type Tarifa struct{ DesdeMetros, HastaMetros int; CostoCentavos int64 }

// Devuelve el costo del envío, o ErrRegla si la distancia excede la cobertura.
func CotizarEnvio(distanciaMetros int, maximoMetros int, tarifas []Tarifa) (int64, error)
```

Reglas de la spec que hay que implementar:

- La cobertura depende **sólo de la distancia vial máxima** desde el local; no hace falta pertenecer al municipio de Rosario.
- Los rangos se evalúan en **metros, sin redondear antes de tarifar**, y no se solapan: cada frontera pertenece a un solo rango. Escribí el test del valor exacto del límite — ahí vive el bug.
- Si la distancia excede el máximo, **no se inventa un costo ni se asume envío gratis**: devuelve `apierr.ErrRegla{Mensaje:"No hacemos envíos a esa distancia. Podés elegir retiro en el local."}`.
- El retiro no tiene cargo.

No integres una API de mapas. Queda fuera del alcance del Parcial I: la distancia llega como parámetro del checkout y se guarda en el pedido. Anotalo en la sección 9 de la spec.

2.3 `estados.go` — la máquina de estados

Es la parte más defendible del módulo: "*sólo se permiten transiciones válidas*" es un criterio explícito de la rúbrica.

```
func PuedeDespachar(p Pedido) error // ENVIO + CONFIRMADO + pago APROBADO
func PuedeEntregar(p Pedido) error  // ENVIO: desde DESPACHADO · RETIRO: desde CONFIRMADO
func PuedeCancelar(p Pedido) error  // PENDIENTE_DE_PAGO o CONFIRMADO. Nunca COMPLETADO
func PuedeCompletarEnEfectivo(p Pedido) error
```

Cada una devuelve `nil` o `apierr.ErrRegla` con un mensaje en castellano que dice **qué hacer**, no sólo qué pasó: "No se puede despachar un pedido que todavía no fue pagado", no "estado inválido".

Reglas que salen de la tabla de la sección 6 de la spec:

Operación	Precondición	Resultado
Despachar	Envío confirmado con pago aprobado	DESPACHADO
Entregar envío	DESPACHADO	COMPLETADO
Completar retiro con MP	Retiro confirmado y pagado	COMPLETADO
Completar retiro en efectivo	Retiro confirmado y no cancelado	COMPLETADO y pago cobrado
Cancelar	Estado permitido	CANCELADO, libera stock

Y una que no se puede olvidar: **no se puede completar una compra web que sigue pendiente de pago en Mercado Pago.**

2.4 Tests

`totales_test.go` , `entrega_test.go` , `estados_test.go` . Mínimo tres casos por regla: el que funciona, el que tiene que fallar y **el borde** — el metro exacto del límite de una tarifa, el pedido con una sola unidad, la cancelación de un pedido ya completado. Ahí vive el bug.

```
git add backend/internal/usecases/pedidos/
git commit -m "feat(pedidos): reglas puras de totales, cotizacion de envio y maquina de estados con tests"
git push
```

BLOQUE 3 — Carrito

SPEC-H07 . `mw.SesionOpcional` : el invitado también compra.

Método y ruta	Qué hace	Errores
GET /api/carrito	Carrito del usuario de la sesión, con unidades, precios y total estimado	401 si manda token inválido
POST /api/carrito/items	Agrega una variante. Si ya está, suma unidades	400, 404, 409
PATCH /api/carrito/items/{id}	Cambia las unidades	400, 404, 409
DELETE /api/carrito/items/{id}	Quita la línea	404 → 204
DELETE /api/carrito	Vacía	204

Reglas de la spec:

- Las unidades son **enteras y positivas**, y no superan la disponibilidad validada contra el servicio de stock. Pedir más que el disponible es **409**, no 400: el request está perfecto, es el estado del sistema el que no lo permite.
- El carrito no reserva unidades ni congela precios.** Los totales se recalculan con el catálogo vigente en cada lectura. Que algo esté en el carrito no garantiza nada hasta el checkout: es justamente por eso que el checkout revalida.

- El carrito del invitado vive en el navegador (`localStorage`), no en la base. En el backend sólo guardás el del cliente registrado. Para el invitado, el checkout recibe los ítems en el body.

```
git add backend/internal/adapters/rest/handlers/pedidos/ \
        backend/internal/adapters/rest/dtos/pedidos/
git commit -m "feat(pedidos): carrito con validacion de unidades contra disponibilidad"
git push
```

BLOQUE 4 — El checkout (la operación que no es un ABM)

SPEC-H13 . Este es el endpoint que se defiende el martes. `mw.SesionOpcional` .

POST `/api/pedidos` → **201** con el pedido creado y su `token_acceso` .

El orden adentro del handler es el de la clase 6, y el orden importa:

1. ¿hay sesión? → opcional acá: el invitado compra igual
2. ¿el body está bien? → 400 validator sobre el DTO
3. ¿las reglas dejan? → 409 stock, cobertura, efectivo habilitado
4. hacer el trabajo → 201

El cuerpo del DTO trae: los ítems (`varianteId` , `unidades`), los datos del comprador (nombre, apellido, correo, teléfono, DNI), `modoEntrega` , `domicilio` y `distanciaMetros` si es envío, y `medioPago` . **No trae precios, ni totales, ni `usuarioId` , ni estados.** Si esos campos existen en el DTO, hay un agujero: con `data: body` cualquiera se crea un pedido pagado de \$0 desde Postman.

Dentro de **una sola transacción de GORM**:

1. Por cada ítem, `lector.VariantePorID(...)` . Si alguna no existe o está inactiva → **404**.
2. `CalcularLineas` con los precios **que devolvió el servidor**, nunca los del body.
3. Si es envío: `CotizarEnvio(distancia, maximo, tarifas)` . Si excede la cobertura → **409** con el mensaje que ofrece retiro.
4. Si es efectivo: sólo se permite con `RETIRO` y si `efectivo_habilitado` está en `true` → si no, **409**.
5. `CalcularTotal` .
6. Crear el `Pedido` con su `numero` , su `token_acceso` (32 bytes de `crypto/rand` , en base64 URL) y los `DetallePedido` con la **copia histórica** de código, nombre y precio.
7. `servicioStock.Reservar(ctx, tx, pedido.ID, items)` . Si no alcanza → **409 enumerando qué falta**, con el mapa variante → disponible en `detalles` . El criterio de aceptación pide enumerar: un booleano no sirve, y meter la lista adentro del string del mensaje obliga a la pantalla a parsear castellano.
8. Estado inicial según el medio de pago:
 - Mercado Pago → `PENDIENTE_DE_PAGO` / `PENDIENTE` , con `vence_en = creado_en + 15 min` (`cfg.PaymentReservationTTL`).
 - Efectivo con retiro → `CONFIRMADO` / `PENDIENTE` , **sin vencimiento automático**.
9. Si algo de todo esto falla, la transacción revierte entera: no puede quedar un pedido creado con el stock sin reservar, ni al revés.

El error más común de este endpoint es devolver **500 donde iba un 409**. Desde afuera los dos se ven igual de rotos y sólo el número los distingue. Ningún `throw` propio adentro de un `try` : los errores esperados se devuelven como valor

y el handler los traduce; sólo lo inesperado sube al `apierr.Responder` final.

```
git add backend/internal/adapters/rest/handlers/pedidos/ \
        backend/internal/usecases/pedidos/
git commit -m "feat(pedidos): checkout transaccional que revalida precios y stock en el servidor"
git push
```

BLOQUE 5 — Consulta del pedido

SPEC-H16 .

Método y ruta	Quién	Qué hace
GET /api/pedidos	con sesión	Los pedidos del usuario del contexto , paginados
GET /api/pedidos/{id}	con sesión	Detalle, buscado con <code>id AND usuario_id</code>
GET /api/pedidos/publico/{token}	público a propósito	El pedido del enlace privado del invitado

Las dos que deciden la nota:

- `GET /api/pedidos/{id}` con el id de un pedido ajeno devuelve **404**, no 403. El problema es la pertenencia, no el rol: el pedido del vecino, para vos, no existe. Un 403 confirmaría que existe, que en un listado de ids ya es información gratis.
- `GET /api/pedidos/publico/{token}` es público **a propósito**, y por eso tiene su **propio Select**, más chico: productos, cantidades, precios, estado y datos de entrega, **sin el `usuario_id`, sin el correo del comprador si no es necesario y sin ids internos**. Reusar el `Select` de las pantallas propias sería el bug, porque el día que alguien le agregue un campo, ese campo se publica en internet sin que nadie lo decida. Dejá el comentario diciendo exactamente eso.

El `token_acceso` es un valor de 32 bytes aleatorios: no adivinable, y nunca se escribe en un log.

```
git add backend/internal/adapters/rest/handlers/pedidos/
git commit -m "feat(pedidos): consulta propia con 404 por pertenencia y enlace publico con select reducido"
git push
```

BLOQUE 6 — Las transiciones (la parte que no es CRUD)

SPEC-H17 y SPEC-H18 . Todo bajo `mw.RequierePermiso("pedidos.gestionar")` .

Fijate en la forma de las rutas: **el verbo se convierte en sustantivo**, que es lo que las mantiene REST. No es `PATCH /api/pedidos/{id}` con `{"estado":"DESPACHADO"}` en el body, porque ahí es el cliente el que decide la transición y el servidor obedece: nada impide saltarse un paso. Con el sub-recurso, el cliente **pide que ocurra la operación** y el servidor decide si corresponde. Además cada transición tiene su propio permiso.

Método y ruta	Qué hace	Errores
GET /api/admin/pedidos	Listado con filtros por estado y fecha	401, 403

Método y ruta	Qué hace	Errores
GET /api/admin/pedidos/{id}	Detalle completo con comprador y pago	401, 403, 404
POST /api/admin/pedidos/{id}/despacho	Marca despachado	401, 403, 404, 409
POST /api/admin/pedidos/{id}/entrega	Marca completado	401, 403, 404, 409
POST /api/admin/pedidos/{id}/cobro-efectivo	Registra cobro y entrega en una sola acción	401, 403, 404, 409
POST /api/admin/pedidos/{id}/cancelacion	Cancela, libera stock	400, 401, 403, 404, 409

Cada uno hace exactamente lo mismo: busca el pedido (404 si no está), llama a la función pura de `estados.go` (409 con su mensaje si no corresponde), y recién entonces, **dentro de una transacción**, cambia el estado.

La cancelación es la más delicada y la que más se pregunta:

- Libera el stock con `servicioStock.Liberar(ctx, tx, pedidoID)`.
- Es **idempotente**: cancelar dos veces no devuelve stock dos veces. Lo garantiza el chequeo de estado más la idempotencia de `Liberar`.
- Si el pedido estaba pagado, la respuesta **dice explícitamente que el reintegro se hace fuera del sistema**: `{"error":null,"aviso":"El reintegro debe realizarse manualmente en Mercado Pago. Este sistema no ejecuta devoluciones."}`. Lo pide la spec y es la clase de detalle que separa un TP de un producto.
- Guarda responsable (del contexto, **nunca del body**), fecha y motivo.
- **No** se puede cancelar un pedido `COMPLETADO`.

Ninguno de estos endpoints cambia productos, cantidades, precios ni domicilio del pedido: la copia histórica es inmutable.

```
git add backend/internal/adapters/rest/handlers/pedidos/
git commit -m "feat(pedidos): transiciones de despacho, entrega, cobro en efectivo y cancelacion"
git push
```

BLOQUE 7 — Pruebas y cierre

7.1 docs/api/pedidos.http

@base en `http://localhost:8080` con la línea de producción comentada al lado. Por cada endpoint, **tres requests**: la que anda, la del body inválido (400, no 500) y la del recurso que no existe (404). Arriba de cada una, el status esperado escrito.

Y estas seis, que son las que hacen la demo del martes:

```

### el flujo feliz completo → 201 con el pedido y su token
POST {{base}}/api/pedidos

### pidiendo más unidades de las que hay → 409 ENUMERANDO qué falta
POST {{base}}/api/pedidos

### mandando el precio en el body → el pedido se crea con el precio del SERVIDOR
POST {{base}}/api/pedidos

### un envío fuera de la cobertura → 409 ofreciendo retiro
POST {{base}}/api/pedidos

### con el token de un cliente, pidiendo el pedido de OTRO → 404
GET {{base}}/api/pedidos/{{idDeUnPedidoAjeno}}
Authorization: Bearer {{tokenCliente}}

### despachar un pedido que todavía no fue pagado → 409
POST {{base}}/api/admin/pedidos/{{idPendiente}}/despacho
Authorization: Bearer {{tokenDueno}}

```

Los tokens van en `docs/.env` (que está en el `.gitignore`), nunca commiteados. Mirá el status, no el cuerpo.

Después del merge, descomentá la `@base` de producción y corré el archivo entero. **Un endpoint que anda en localhost y no en la URL pública, para esta materia, no existe.**

7.2 El flujo principal, de punta a punta

Dejá escrito en `docs/api/pedidos.http`, arriba de todo, el recorrido completo numerado, con los ids del seed, para poder correrlo de un tirón el martes:

1. GET /api/catalogo/variantes?q=...	ver el catálogo
2. POST /api/pedidos	crear el pedido (201, reserva stock)
3. POST /api/pedidos/{id}/pago	iniciar Mercado Pago (devuelve init_point)
4. POST /api/pagos/webhook	simular la aprobación → pedido CONFIRMADO
5. GET /api/pedidos/publico/{token}	el invitado consulta su pedido
6. POST /api/admin/pedidos/{id}/despacho	→ DESPACHADO
7. POST /api/admin/pedidos/{id}/entrega	→ COMPLETADO

Ese recorrido **es** el flujo de negocio real de punta a punta que pide el núcleo obligatorio. Tiene que correr entero contra producción.

7.3 Spec

Actualizá en `docs/specs.md` lo que efectivamente se implementó y movelo a la sección 9 lo que no: promociones, cupones, facturación, favoritos, avisos de reposición y geocodificación con API de mapas quedan **fuera del alcance de esta entrega**. Lo que no está recortado por escrito, en la defensa cuenta como prometido y no entregado.

```
git add docs/api/pedidos.http docs/specs.md
git commit -m "docs(pedidos): recorrido del flujo principal, requests de prueba y recorte de alcance"
git push
gh pr create --base develop --title "Carrito, checkout y pedidos" --body "Carrito, checkout
transaccional con revalidación de precios y stock, copia histórica del pedido, enlace privado para
invitados y las transiciones de despacho, entrega y cancelación."
```

Definición de terminado

No te detengas hasta que todo esto sea cierto:

- ☐ `go build ./... && go vet ./... && go test ./...` en verde.
- ☐ La migración de pedidos aplicada en la base de la nube.
- ☐ El flujo completo corre de punta a punta **contra producción**: catálogo → pedido → pago →
- ☐ Un checkout que manda `precioUnitario: 1` en el body crea el pedido con **el precio real**.
- ☐ Un checkout que pide más unidades de las que hay devuelve **409 enumerando qué falta**.
- ☐ Un envío fuera de la cobertura devuelve **409 ofreciendo retiro**, no un costo inventado.
- ☐ `GET /api/pedidos/{id}` con el id de un pedido ajeno devuelve **404**.
- ☐ Despachar un pedido no pagado devuelve **409**, no 500.
- ☐ Cancelar dos veces no devuelve stock dos veces.
- ☐ El enlace público devuelve menos campos que la consulta propia.
- ☐ `docs/api/pedidos.http` corre entero contra producción.
- ☐ Todo pusheado, con commits repartidos y a tu nombre.

Si te trabás

- Si el lector de Genaro o el servicio de stock de Agustín todavía no están en `develop`, declará **tu propia interfaz** en `backend/internal/core/ports/out/` y trabajá contra un stub en memoria. Cuando lleguen los reales, sólo cambia qué se inyecta. El proyecto es hexagonal exactamente para esto.
- Si una migración falla, **nunca** edites una ya aplicada ni corras un reset contra la base de la nube. Se corrige con una migración nueva.
- Si algo del alcance no llega, priorizá en este orden: checkout, consulta del pedido, transiciones, carrito. El checkout es el endpoint que se defiende.

Lo que te van a preguntar el martes

- Mostrame qué pasa si le pido este pedido con la cuenta de otro.
- ¿De dónde sale el precio del pedido? Mostrame qué pasa si lo mando en el body.
- ¿Por qué el detalle del pedido guarda el nombre y el precio, si eso ya está en la variante?
- ¿Qué pasa si dos personas compran la última unidad al mismo tiempo?
- ¿Por qué `POST /api/admin/pedidos/{id}/despacho` y no `PATCH` con el estado en el body?
- ¿Qué pasa si cancelo dos veces el mismo pedido?

7. ¿Cuál es el flujo de negocio de punta a punta de su sistema, y cuál es la operación que no es un ABM?

8. ¿Por qué 404 y no 403 cuando el pedido es de otro?